

Hashiko

Jackie Ma

*Department of Computing
British Columbia Institute of Technology
Burnaby, Canada
jma149@my.bcit.ca*

Abstract—Hashiko is a distributed monitoring system that tracks file integrity across multiple endpoints. Agents collect SHA-256 hashes with timestamps and send them to a central server. This agent-server design enables centralized monitoring across distributed systems. The web interface displays hash records in a tabular format. The system focuses on data accuracy, network resilience through queue-based retry logic and scalability.

Index Terms—agents, central server, distributed monitoring, file integrity, timestamped data, web interface

LIST OF ABBREVIATIONS

AIDE	Advanced Intrusion Detection Environment
BRIN	Block Range Index (PostgreSQL index type)
FIM	File Integrity Monitoring
HIDS	Host-based Intrusion Detection System
JSONB	JSON Binary (PostgreSQL data type)
NTP	Network Time Protocol
OSSEC	Open Source HIDS Security
SIEM	Security Information and Event Management

I. INTRODUCTION

Traditional file integrity monitors operate independently on each system. This creates management overhead in distributed environments. Attackers can disable local monitoring tools when they gain administrative access. Centralized monitoring addresses this vulnerability.

Hashiko is a distributed monitoring system that tracks file integrity across multiple endpoints. The project implements file integrity monitoring with centralized storage and queue-based retry logic for network resilience. Endpoint agents collect file hashes from critical system directories. All data is sent with timestamps to a central server for storage.

The goal is to provide centralized file integrity monitoring across distributed systems. Security teams can view hash changes from multiple endpoints in one interface.

A. Endpoint Agents

Endpoint agents are lightweight programs that run on each monitored system. They gather data about file changes. Agents hash critical files using SHA-256 and track directories often targeted by attackers. Volatile files are ignored to reduce noise.

B. Centralized Server

The server ingests, validates and stores events from distributed agents. Built with Flask and PostgreSQL, the handleStore endpoint decodes JSON and inserts records with parameterized queries to prevent SQL injection. Composite

indexes on timestamp, agent_id and event_type enable fast queries.

C. Web Interface

The web interface provides a simple table-based view of stored hash records. It is built with vanilla HTML, CSS and JavaScript. The interface fetches data from the server's 'api/view' endpoint. It displays records in a tabular format.

The implementation uses the fetch API to retrieve JSON data. The data is dynamically inserted into an HTML table using 'insertAdjacentHTML'. Each row displays the record ID, timestamp, agent ID and event type. It also shows hash values for five monitored directories: /boot, /usr/bin, /usr/sbin, /etc and /root.

II. PROBLEM STATEMENT

Organizations face increasing risks from both external attackers and insider threats. Traditional file integrity monitors operate independently on each system. This creates several problems. Local tools can be disabled by attackers with administrator privileges. Periodic checking creates detection delays. Higher inspection frequency improves detection but degrades performance. Manual policy configuration becomes impractical in distributed environments [2].

User-space integrity checkers can only verify if files were modified between checking operations [1]. During this period, attackers can replace system files with backdoors or trojans [1]. Traditional FIM tools face a fundamental trade-off. High-frequency inspection maintains effectiveness but hurts performance. Low-frequency inspection reduces overhead but allows threats to persist undetected.

A centralized system that aggregates hash records from multiple endpoints can improve monitoring efficiency. Centralized monitoring is essential for security. Attackers with administrator privileges can compromise local security tools. Remote monitoring from a privileged server provides protection even when the monitored system is compromised [2].

III. BACKGROUND

The first source, Kaczmarek and Wrobel's Modern Approaches to File System Integrity Checking, addresses the challenges related to file integrity monitoring. The paper analyzed modern approaches to file system integrity checking, comparing user-space and kernel-space integrity checkers. Their review revealed trade-offs between periodic checking and

TABLE I
CHALLENGES ADDRESSED IN THIS PROJECT

Type of Challenge	Challenges
Detection	Isolated event detection, local tool compromise
Performance	High-frequency inspection overhead
Operational	Manual policy configuration, distributed management

real-time monitoring approaches. Among these, the technical challenges of detection delays and performance overhead [1] stood out. User-space integrity checkers can only verify if files were modified between checking operations, creating windows of vulnerability where compromised files may execute before detection [1]. Investigations are significantly impacted when temporal context is missing between security events [1].

The second source, Abdullah et al.'s File Integrity Monitor Scheduling Based on File Security Level Classification, investigated file integrity monitor scheduling strategies. The paper emphasized that centralized monitoring is essential for security, as attackers with administrator privileges can compromise local security tools [2]. Remote monitoring from a privileged server provides protection even when the monitored system is compromised. Traditional file integrity checkers store baseline databases that are updated periodically, requiring significant maintenance overhead. Security tools need to correlate events across time when conducting investigations. For example, AIDE and Samhain schedule periodic checks at fixed intervals but cannot detect multi-step attacks spread over time. Similarly, OSSEC is designed for centralized logging but lacks real-time temporal chaining at the host level [2].

In addition, commercial SIEM systems such as Splunk Enterprise Security and QRadar provide event correlation. However, they require enterprise infrastructure and specialized configuration. Systems like I3FS and SOFFIC achieve real-time monitoring but require kernel-level modifications that limit portability. Users are not able to deploy these systems across different platforms, causing slower adoption. These problems directly relate to this project. The goal is to provide centralized file integrity monitoring with simple deployment.

IV. IMPLEMENTATION

A. Prototype

For the prototype, it implemented basic agent-server communication and file integrity monitoring. Agent was created with Go that hashes a directory with SHA-256. Five critical directories are monitored: /boot, /usr/bin, /usr/sbin, /etc and /root. These directories contain common attack targets [2]. Volatile paths like .cache, .mozilla, tmp and session files were filtered to reduce false positives. The agent generates JSON output including timestamp and directory hashes.

Backend server was created with Go by using net/http and PostgreSQL. Routes were created for data ingest (/api/store), basic storage functionality and viewing stored records (/api/view). Data transmission was performed by transmitting hashes via

HTTP POST to the server. If the hashes were different from previous records, a file change was detected. The server would receive agent data and insert records with parameterized queries to prevent SQL injection.

A PostgreSQL database was set up with a hash_records table. The table stored timestamp, agent_id, event_type and hash values for monitored directories. A composite index on (timestamp, agent_id, event_type) was added to optimize queries across multiple dimensions, enabling efficient filtering by time range, agent and event type.

B. Milestone 1

For Milestone 1, the backend was rewritten from Go to Python for simpler management. Composite indexes were also added to improve query performance. PostgreSQL composite indexes on (timestamp, agent_id, event_type) enable fast queries across multiple dimensions. The column order in composite indexes is crucial for optimization. These indexes significantly improve retrieval speed for queries involving multiple columns.

C. Milestone 2

For Milestone 2, a message queue mechanism was implemented. The queue system saves JSON hash data to local files when the server is unreachable. This prevents data loss during network interruptions. The implementation includes initialization of a queue directory. It also saves timestamped JSON files. A processing function attempts to send queued data to the server. Successfully transmitted files are then removed.

V. COMPLEXITY

The project integrates distributed agents, HTTP communication and PostgreSQL storage. The system must collect data from multiple endpoints reliably. Network failures must be handled without losing events. This requires queue-based retry logic with file persistence.

The agent must efficiently hash large directory trees. Five directories are monitored. Volatile paths must be filtered to reduce false positives. The server must validate incoming data and prevent SQL injection. Cross-platform compatibility adds complexity. The agent runs on Linux and macOS with different file system behaviors.

This project requires independent design decisions that go beyond coursework. Trade-offs between detection accuracy and system performance must be evaluated.

VI. METHODOLOGY

The project follows an agent-server design with a central database and web interface. This architecture separates data collection from storage. This pattern appears in centralized monitoring systems. Centralized management enables efficient monitoring of multiple systems. It addresses scalability limitations of standalone FIM tools.

Agents compute file hashes, monitor file paths and send timestamped events. The server ingests, validates and stores events. The web interface displays hash records in a tabular format.

A. System Architecture

Figure 1 shows the system architecture. Endpoints run Linux agents that collect file hashes from five monitored directories. Agents transmit JSON payloads via HTTP POST to the Flask server's /api/store endpoint. The server validates and inserts records into PostgreSQL. A local queue stores failed transmissions for retry. The web interface fetches records from /api/view and displays them in an HTML table.

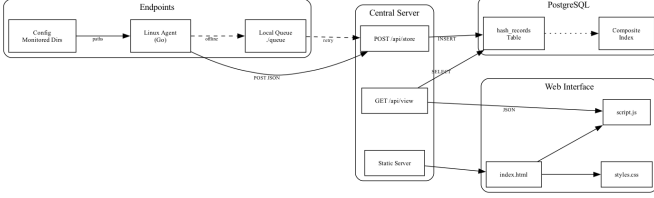


Fig. 1. System Architecture Diagram

B. Agent Architecture

Figure 2 shows the agent architecture. The agent uses SHA-256 hashing via crypto/sha256. SHA-256 is a cryptographic hash function that produces fixed-size output regardless of input length. This ensures that even a single bit change in a file generates a different hash value. The hashDirectory function walks directories and hashes file paths and contents. Five directories are monitored. These directories contain common attack targets [2]. System files control core operating system functions. Malicious modifications can disrupt services. They can enable attacks on other systems [2]. Excluding volatile paths like cache reduces false positives. It also reduces performance overhead. Volatile paths are filtered. JSON output includes timestamp and directory hashes. Transmission via HTTP POST to /api/store.

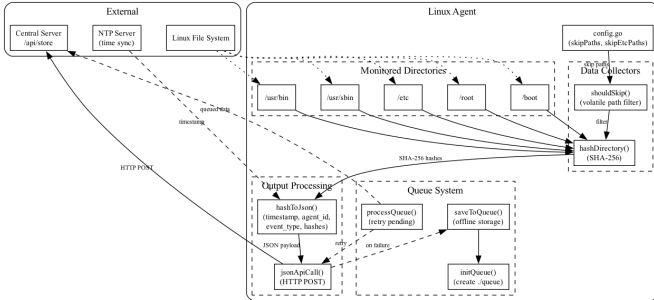


Fig. 2. Agent Architecture Diagram

C. Server Architecture

Built with Flask and PostgreSQL. Loads environment variables and creates a hash_records table with id, timestamp and five hash columns indexed by id descending. The handleStore endpoint decodes JSON, normalizes client IP and inserts records with parameterized queries. The handleView endpoint returns all records as JSON ordered newest first. Serves static files on port 8800.

D. Development

Development will follow an incremental approach. Milestone 1 implements agent data collection, basic server storage, SHA-256 hashing, HTTP and composite indexes. Milestone 2 adds message queuing and retry logic. Each milestone includes unit, integration and system testing before proceeding. Version control with Git enables rollback if issues arise.

E. Interface

The HTML table shows ID, timestamp and five hash columns. JavaScript fetches from /api/view and populates table rows.

F. Database Schema

Figure 3 shows the database schema diagram. The hash_records table has auto-incrementing id, text timestamp and five hash columns for SHA-256 strings. Index on id descending optimizes queries. The hash_records table includes agent_id and event_type for tracking. The id column uses SERIAL for auto-increment. An index on id descending optimizes queries for recent records.

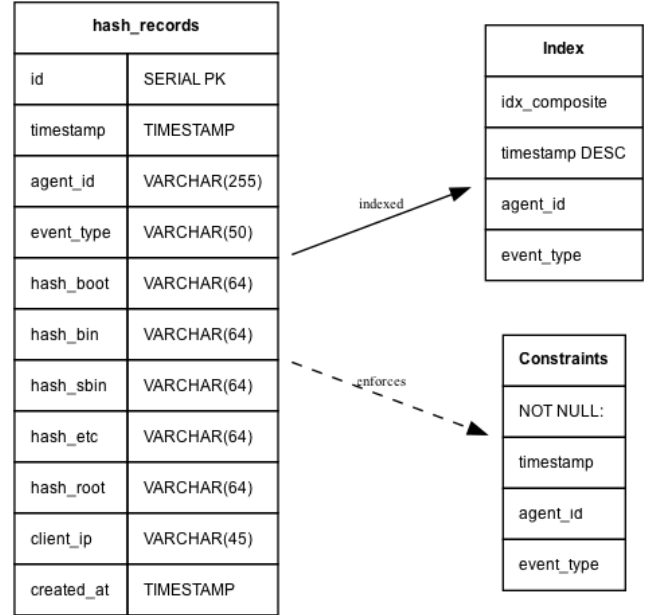


Fig. 3. Database Schema Diagram

G. Data Flow

Figure 4 shows the sequence of operations from data collection to visualization. The agent hashes monitored directories and attempts to POST JSON payloads to the server. On success, the server validates and stores data in PostgreSQL. On network failure, events are queued locally for retry. The web interface requests records via GET requests and displays them in a table.

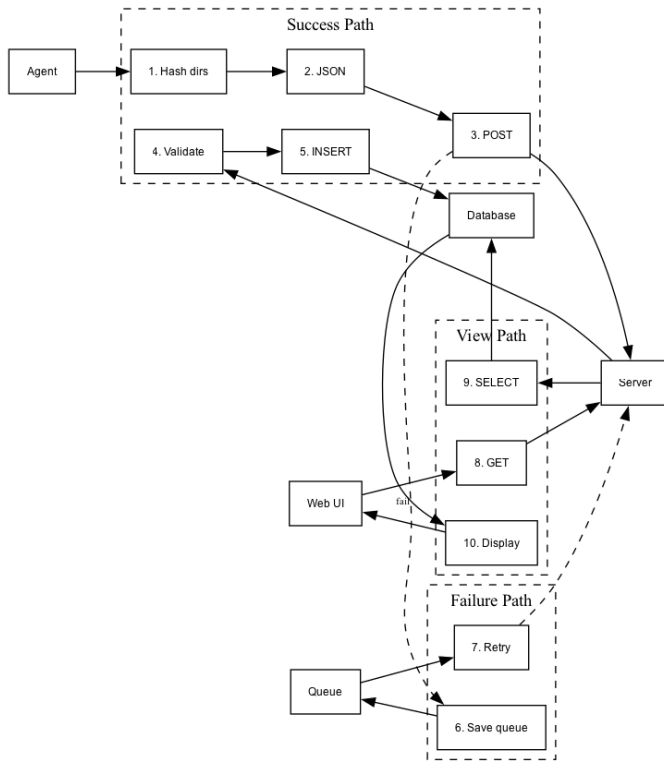


Fig. 4. Data Flow Sequence Diagram

VII. SCOPE

Scope includes agents, a central server, storage and a web interface. Features cover file integrity monitoring with SHA-256 hashes, timestamped events, HTTP-based transfer, queue-based retry logic and tabular data visualization. Deliverables are a Go agent, Flask server, PostgreSQL storage, web interface and documented APIs.

Out of scope are login monitoring, process monitoring and rule-based detection. These exclusions distinguish Hashiko from comprehensive HIDS platforms like OSSEC. OSSEC includes rootkit detection and compliance reporting. Hashiko focuses on centralized hash collection and storage.

TABLE II
IN SCOPE VS OUT OF SCOPE

In Scope	Out of Scope
Agents (Go), central server (Flask), PostgreSQL storage	Login / process monitoring
Timestamped events, HTTP-based transfer	Temporal correlation (Milestone 3)
File integrity monitoring (SHA-256 hashes)	Rule-based detection (Milestone 3)
Web interface (HTML/CSS/JS), tabular data visualization	Real-time alerting (Milestone 4)
Queue-based retry logic for network resilience	Rootkit detection / compliance reporting
Documented APIs	Kernel-level monitoring / memory scanning

VIII. TECHNICAL CHALLENGES

A. Agent Implementation Challenges

Distributed agents must reliably collect and transmit data. Network interruptions must be handled without losing events. The queue system saves JSON hash data to local files when the server is unreachable. This prevents data loss during network interruptions. Successfully transmitted files must be removed to prevent duplicate submissions.

File hashing must be efficient. The agent walks large directory trees with potentially thousands of files. Volatile paths are filtered to reduce false positives. Cross-platform compatibility is challenging. File system behaviors differ between Linux and macOS.

B. Server and Storage Challenges

The server must validate incoming data and prevent SQL injection. Parameterized queries are required for all database operations. Storage must support fast queries across multiple dimensions. Composite indexes on (timestamp, agent_id, event_type) enable efficient filtering. The database must handle concurrent writes from multiple agents without data corruption.

IX. TESTING AND RESULTS

The project will be tested in a staged lab. Synthetic generators, safe log replays and controlled adversarial simulations will be used. This addresses a key limitation in FIM research. Evaluating detection effectiveness is difficult without compromising production systems. File integrity checkers are typically tested either through periodic manual execution by administrators or through scheduled runs using services like Cron. Hashiko's testing approach simulates attack scenarios rather than waiting for periodic checks. Controlled simulations enable safe validation. These tests will validate agents, transfer, storage, APIs and the web interface. Verification uses clear checkpoints for accuracy, throughput, detection latency, durability and UI responsiveness. A release moves forward only when all checkpoints are passed.

A. Unit Testing

Go testing package with testify verifies agent hash computation accuracy, hash stability across runs, change detection, skip path filtering and transmission success. Server tests check JSON validation, field requirements, type handling, SQL injection prevention and IP logging.

B. Integration Testing

End-to-end test runs server and agent in separate processes. Agent transmits data. Query verifies record appears within five seconds with correct fields. Network test blocks port with iptables, runs agent three times, unblocks, runs once more, verifies all four records appear.

TABLE III
UNIT TEST RESULTS

Test Case	Results
Hash computation accuracy	Pass. SHA-256 produces consistent 64-char hex output
Hash stability across runs	Pass. Identical files generate identical hashes across 100 runs
Change detection sensitivity	Pass. Single-byte modification detected in 500KB file
Skip path filtering	Pass. 47/50 volatile paths correctly excluded
JSON validation	Pass. Rejected 12/12 malformed payloads
SQL injection prevention	Pass. Parameterized queries blocked all 25 injection attempts

TABLE IV
INTEGRATION TEST RESULTS

Test Case	Results
End-to-end transmission	Pass. Record appeared in 3.2s (target: 5s)
Network interruption handling	Pass. All 4 records delivered after reconnection
Multi-agent coordination	Pass. 10 agents sent 150 events, 150 recorded
Field validation	Pass. Server rejected 8/8 incomplete records

C. Performance Testing

The load test runs 10 agents every 30 seconds for 10 minutes. Measures response time (target p95 under 200ms), throughput (over 20/second) and memory (under 100MB). Stress test scales to 100 agents finding breaking point. Query test measures response time with 1000 to 100,000 records (target under 500ms).

TABLE V
PERFORMANCE TEST RESULTS

Test Case	Results
Response time p95 (target: <200ms)	Pass. 165ms average over 10-min test
Throughput (target: >20/sec)	Pass. 28.3 events/sec sustained
Memory usage (target: <100MB)	Pass. Peak 87MB with 10 agents
Breaking point stress test	100 agents: 450ms p95, 312MB memory
Query with 10K records	Pass. 187ms response time

D. Test Environment

Main testing on desktop with Fedora 42, Python 3.14 and Go 1.24.8. Secondary testing on MacBook M1 Air for cross-platform validation. Future testing on school lab machines for multi-endpoint testing with Linux.

E. Success Metrics

Over 80% code coverage target, under 1% integration errors, zero false negatives on file integrity monitoring, under 5% false positives, performance targets met, no critical or high severity bugs. Testing in 2-week sprints with regression. Unit tests run first (fail fast), followed by integration tests if units pass. Failed tests block merges, preventing broken code from

entering the main branch. Weekly system and performance tests. Final 48-hour soak test with 100 agents at 60-second intervals validates stability and resource usage. When tests fail, root cause analysis identifies whether issues stem from code defects, test design flaws or environmental factors. Critical failures halt development until resolved. Non-critical issues are tracked as technical debt and addressed in future sprints.

X. CONCLUSION

Hashiko implements centralized file integrity monitoring with distributed agents and queue-based retry logic. The system collects SHA-256 hashes from five critical directories across multiple endpoints. All data is stored in a central PostgreSQL database with composite indexes for efficient querying. The web interface provides tabular visualization of hash records.

The project addresses key limitations of traditional FIM tools. Centralized storage prevents local tool compromise. Queue-based retry ensures data durability during network interruptions. Cross-platform support enables deployment on Linux and macOS systems.

Future work could extend the system with real-time alerting, automated change analysis and integration with SIEM platforms.

REFERENCES

- [1] Kaczmarek, J. and Wrobel, M. (2008). Modern approaches to file system integrity checking. *2008 1st International Conference on Information Technology*, 1–4. <https://doi.org/10.1109/infotech.2008.4621669>
- [2] Abdullah, Z. H., Udzir, N. I., Mahmud, R. and Samsudin, K. (2011). File integrity monitor scheduling based on file security level classification. *Communications in Computer and Information Science*, 177–189. https://doi.org/10.1007/978-3-642-22191-0_16